

Phase 2-Full: STM + Round + Importance 구현 계획

목표: 현재 baseline (LTM + stateless memory_window 함수)을 3-tier 메모리 시스템 (LTM SSD / STM RAM cache / KV-cache) + 라운드 비동기 처리 + Topic importance 4 작용으로 확장. Phase 4 sanity 결과의 multi-session 약점(0.20)이 baseline의 **stateless re-selection**과 **fixed budget**에서 기인함이 확인됨 → 본 설계는 정확히 그 두 axis를 풀어냄.

Hi-EM의 winning thesis(적은 token으로 baseline 동등)를 회복하는 path. 단 본 설계 도입 후에도 RAG 못 넘으면 정직 reframing.

0. 현재 구현 vs 목표 (사용자 표 그대로)

0.1 메모리 계층

항목	현재 (src/hi_em/)	목표
LTM 위치	data/ltm/<conv>/{state.json, *.jsonl}	동일
LTM 내용	turn 원문 + topic_id + turn_id + embedding + topic centroid	동일
LTM 쓰기 시점	매 턴 sync	동일
STM 존재	× (memory_window 함수만)	✓ MemoryWindow 클래스 (RAM in-process)
STM 내용	—	importance $\geq$ threshold인 topic의 turn 전체
STM 상태 보존	—	라운드 사이 유지, 매 턴 누적/eviction
Eviction	—	가득 차면 lowest importance topic 통째 제거

0.2 매 턴 처리

항목	현재	목표
분절 신호	sCRP centroid distance ✓	동일

항목	현재	목표
Retrieval source	LTM stateless 재선별	STM 우선, miss 시 LTM→STM 승격
Retrieval 단위	topic top-3 × 마지막 5 turn	STM 내 topic 전체
Cache 디코딩	×	✓ KV-cache 재사용 (vLLM prefix cache)
Cold start	빈 LTM이면 no prefill	LTM 직접 디코딩
응답 후	LTM append	동일

0.3 라운드 처리 (신규)

항목	현재	목표
Round 개념	×	✓ 10 turn = 1 round
실행 시점	—	IDLE 시간, async
LTM 재구성	append-only	topic_id 기준 정렬 (단 turn_id 는 보존)
STM 갱신	—	importance 기반 promotion

0.4 Topic Importance (신규)

작용	효과	공식 (잠정)
강화 (turn 수)	turn 많을수록 ↑	$\log(1 + n_t)$
강화 (빈도)	자주 언급될수록 ↑	round별 mention count의 EMA
망각 (recency)	오래될수록 ↓	$\exp(-\lambda_r \cdot (\text{round}_{\text{now}} - \text{round}_{\text{last}}))$
연결 (인접/통합)	인접 topic + 통합 신호 강한 topic의 importance 가중 합	$\sum_j w_{ij} \cdot I_j$

종합:

$$I_t = \alpha_1 \log(1 + n_t) + \alpha_2 \text{EMA}(\text{freq}) + \alpha_3 \exp(-\lambda_r \Delta \text{round}) + \alpha_4 \sum_j w_{tj} I_j$$

가중치 $\alpha_{1..4}$ 초기값은 모두 1.0 (uniform), Phase 4 sweep 결과로 튜닝.

1. 구현 단계 (Step 별)

Step P2F-1: `topic_importance.py` (단위 함수, 1일)

- `compute_importance(topics_state, round_now, mention_log) -> dict[topic_id -> float]`
- 4 작용 각각 분리 함수 -> 종합 -> unit tests 8~10개
- 검증: 4 작용 각각 isolation test + edge case (빈 history / 최근 1 turn / 통합 신호 없음)

Step P2F-2: `MemoryWindow` class (RAM 캐시, 2일)

- 모듈 `src/hi_em/memory_window.py` 확장 (현재 함수만 -> class로)
- API:

```
class MemoryWindow:
    def __init__(self, max_topics: int, max_turns: int): ...
    def get(self, topic_id: int) -> list[dict] | None # cache hit/miss
    def promote(self, topic_id: int, turns: list[dict]) -> None # LTM -> STM
    def evict_lowest_importance(self, importance: dict[int, float]) -> int | None
    def all_turns(self) -> list[dict] # current STM 전체 turn (chronological)
    def topics_in_cache(self) -> set[int]
```

- 기존 `select_memory_window` 함수 deprecate (Phase 2 baseline 평가용으로 유지하되 internal로)
- 검증: cache hit/miss, eviction policy, capacity boundary, 라운드 사이 state 유지

Step P2F-3: `RoundProcessor` (3일)

- 모듈 `src/hi_em/round_processor.py`
- API:

```
class RoundProcessor:
    def __init__(self, ltm: LTM, stm: MemoryWindow, threshold: float, alpha: list[float], lambda_r: fl
    def process(self, conv_id: str) -> RoundResult
        # 1. mention log 갱신 (이번 라운드의 turn 분포)
        # 2. compute_importance (모든 topic)
```

- # 3. STM promotion: $\text{importance} \geq \text{threshold}$ 인 topic 모두 STM에 (없으면 promote)
 - # 4. STM eviction: capacity 초과 시 lowest importance 제거
 - # 5. (선택) LTM 재구성: state.json topic_id 기준 정렬 (단 jsonl은 append-only 유지)
- async 실행: asyncio 또는 concurrent.futures. 단일 process이므로 단순 thread 충분.
 - 트리거: HiEM.handle_turn에서 turn count % 10 == 0일 때 round_processor.process 호출 (background thread).
 - 검증: 10 turn 후 자동 발동 / mention log 정확성 / promotion threshold 동작 / eviction 후 STM 크기 보장

Step P2F-4: HiEM.handle_turn 수정 (1일)

- 매 턴 흐름 변경:

```
def handle_turn(self, user_text: str) -> str:
    q = self._encoder.encode([user_text])[0]
    topic_id, is_boundary = self._segmenter.assign(q)

    # NEW: STM 우선
    stm_turns = self._stm.get(topic_id)
    if stm_turns is None:                                     # cache miss
        ltm_turns = self._ltm.load_turns(self.conv_id, topic_id=topic_id)
        self._stm.promote(topic_id, ltm_turns)
        stm_turns = ltm_turns

    # NEW: STM 전체 chronological prefill (cache 디코딩 가능)
    prefill = self._stm.all_turns()
    messages = [...] # 동일

    response = self._llm.chat(messages, model=self._model, **self._llm_kwargs)
    self._ltm.append_turn(...) # user
    self._ltm.append_turn(...) # assistant

    # NEW: 라운드 트리거
    if self._next_turn_id % (2 * 10) == 0: # 10 user+assistant pair
```

```
self._round_processor.process_async(self.conv_id)

return response
```

- return_debug 그대로 유지 (Phase 4 metric용 — STM hit/miss flag 추가)
- 검증: 10 unit tests (STM hit / STM miss → LTM promote / 라운드 트리거 / cold start / KV cache 효과는 통합 smoke test)

Step P2F-5: KV-cache 재사용 (2일, 검증 위주)

- vLLM은 **prefix cache 자동** (server side). 같은 prefix가 messages에 등장하면 자동 hit.
- 우리 구현에서 STM이 라운드 사이 유지되면 **prefill prefix가 거의 같음** → vLLM이 자동으로 prefix cache hit. 별도 코드 불필요.
- 단 messages 형식 변동 최소화:
 - system_prompt 고정
 - STM의 turn 순서 고정 (chronological)
 - 매 턴 추가되는 user message만 변경
- 검증: vLLM stats endpoint 또는 응답 시간 측정 — 라운드 진행 시 LLM call latency가 줄어드는지 확인. 줄지 않으면 prefix가 다르게 형성되는 문제.
- **fallback**: vLLM 자동 cache 안 통하면 KV는 우리 영역 밖. document하고 진행.

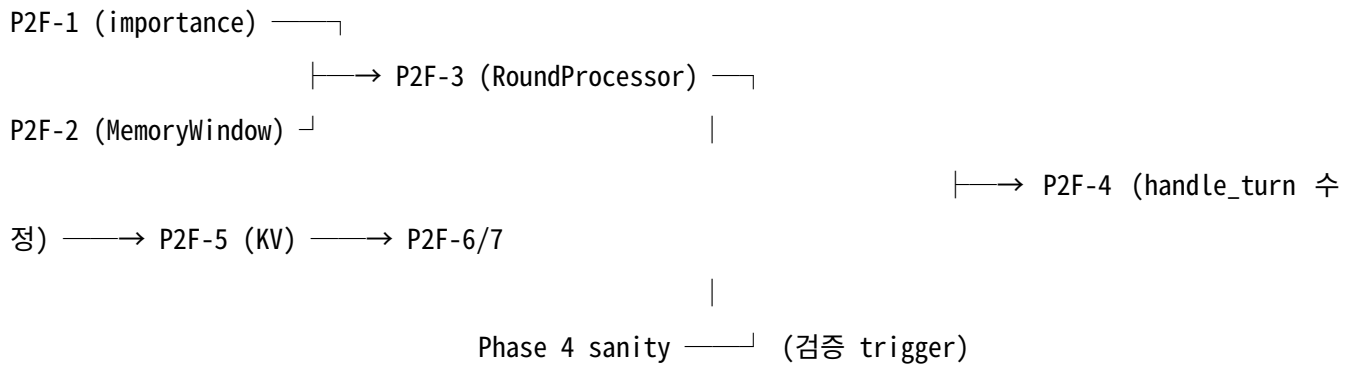
Step P2F-6: 통합 smoke test (1일)

- scripts/smoke_test_full_pipeline.py: A→B→A→B→A 5턴 시나리오, STM hit/miss + 라운드 발동 trace 출력
- 응답 시간 비교: baseline (현재 stateless) vs full pipeline (STM cached)

Step P2F-7: Phase 4 재실행 (반나절)

- run_longmemeval.py의 hi-em method를 새 HiEM으로 자동 교체 (HiEM API 안 바뀌면 코드 변경 없음)
- sanity 30 nothink + sanity 30 think 비교
- 결과로 D/E 결정 (Phase 4 reframing)

2. 의존 그래프



총 추정 시간: $8_{10} \times 1_{10} \times 2_{10}$ (12주).

3. 호환성 / 회귀 위험

3.1 보존해야 할 기능

- `HiEM.handle_turn(user_text)` 시그니처: caller (Phase 4 평가) 변경 없이 동작
- `preload_history(turns)`: 기존 동작 (segmenter 통과 + LTM 저장) 유지하되 STM에는 promote 안 함 (또는 별도 옵션 — caller 결정)
- 58 unit tests: 기존 4 tests (test_orchestrator의 single-turn / first-turn / second-turn / topic-revisit)는 반드시 PASS

3.2 Risk

- STM 상태가 `conv_id` 사이 격리 안 되면 leak: per-conversation STM 인스턴스 강제
- `async` 라운드 처리 race: Python `threading.Lock`으로 STM/LTM 보호 (encoder처럼)
- `importance` 가중치 잘못 설정 시 thrashing: P2F-1 unit test로 monotonicity 검증 + Phase 4 sweep으로 튜닝
- `vLLM prefix cache miss` → 효과 없음: P2F-5 검증 단계에서 latency 측정으로 즉시 발견

3.3 Phase 4 재실행 시 비교 baseline

라벨	의미
hi-em (현재)	stateless re-selection, $k_topics=3 \times k_turns_per_topic=5$

라벨	의미
hi-em-full (P2F 후)	STM cached + round + importance
가설	hi-em-full이 multi-session에서 0.20 → 0.40+

4. 비검증 사항 (P2F 종료 후에도 미해결)

1. **Phase 5 논문 실험·재현**: 본 설계는 평가 인프라 강화. 논문 수치 재현은 Phase 5.
2. **m_cleaned (500 sessions)** 평가: oracle은 짧음. STM의 진짜 가치(긴 history + 토픽 복귀)는 m_cleaned에서만 정량 가능.
3. **importance 4 작용 가중치 최적값**: P2F-7 sweep 후 확정. 본 plan은 baseline 가중치만.
4. **KV-cache 절약 정량**: vLLM endpoint stats 접근 가능해야 측정. 사용자 환경 확인 필요.

5. 다음 단계 (이 plan 채택 시)

1. 본 md를 codex review에 제출 → design 결함 / 누락 / overengineering 지적 받기
2. review 반영 후 P2F-1부터 순차 implement
3. 각 step 별 commit (Phase 2 commit 패턴 유지)
4. P2F-7 (sanity 비교) 결과로 Phase 4 reframing 최종 결정

6. 결정 분기 (사용자 확인 필요)

항목	권고	대안
Round 크기 (turn)	10 (사용자 제안)	5 / 20 / adaptive
STM capacity	max_topics=10, max_turns=200	측정 후 조정
Importance threshold	0.5 (uniform 가중치 가정)	측정 후
Round async lib	threading.Thread (단순)	asyncio (orchestration 늘어나면)

항목	권고	대안
LTM 재구성 범위	state.json만 (jsonl append-only 유지)	jsonl도 재정렬 (디스크 I/O ↑, 지양)
vLLM prefix cache 검증	latency 측정으로 indirect	vLLM endpoint stats API 확인

7. 검증 미해결 (codex review 요청 항목)

1. Topic importance 공식의 monotonicity가 4 작용 모두에서 보장되나? — round count 증가 시 망각이 다른 작용을 압도하는 케이스 등.
2. STM eviction 후 다시 같은 topic 등장 시 promotion 비용 — 매번 LTM에서 다 읽어 RAM에 올리면 효과 무의미. delta 캐싱 필요한가?
3. Round async가 매 턴 흐름과 deadlock 안 함을 보장할 lock 그래프?
4. vLLM prefix cache가 우리 messages 형식에 정말 작동할까? messages list ordering / role 변동에 대한 cache key sensitivity 문서 확인 필요.
5. 본 design이 Phase 4 multi-session 0.20을 실제로 풀 수 있나? — 즉 budget 늘림 + STM 영속화가 정답 누락 문제를 직접 푸는가, 아니면 같은 문제 단순 transparent하게 만드는가?
6. KV-cache 재사용이 정확도 trade-off 없이 token 효율만 주나? — prefix 같으면 LLM 응답도 같음 (deterministic). temperature > 0이면 sampling만 다름.